
RECURSION

A function that solves a problem by calling itself on a smaller version of the same problem



What Is Recursion?

Definition

A recursive function is a function that calls itself, either directly or indirectly, in order to solve smaller instances of the same problem, until the problem becomes small enough to solve directly.

Every day analogies

- **Russian nesting dolls** — Open a doll → find a smaller doll inside → open that → ... → smallest doll has nothing inside (stop).
- **Two mirrors facing each other** — Each reflection contains a smaller reflection of itself, repeating inward.
- **Looking up a word using its own definition** — "To understand recursion, first understand recursion" — until the definition becomes self-evident.

A function that calls itself

```
solve(5)  calls  solve(4)
```

```
solve(4)  calls  solve(3)
```

```
solve(3)  calls  solve(2)
```

```
solve(2) → base case  
reached
```

Each call hands off a smaller version of the problem to the next call.

Every Recursive Function Needs Two Parts

1 Base Case

The simplest version of the problem — small enough to answer directly, with NO further recursive call. This is what stops the recursion.

Without it → infinite recursion → stack overflow

2 Recursive Case

Breaks the problem into a smaller version of the same problem, and calls the function again on that smaller version.

Must move the input closer to the base case every time

General Skeleton

```
return_type solve(problem) {  
    if (problem is small enough) {  
        return direct_answer;  
    }                                     // base case  
    return combine(solve(smaller_problem));  
}                                     // recursive case
```

WORKED EXAMPLE 1

Factorial of N

Mathematical definition

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$$

Base case: $0! = 1$

Recursive case: $n! = n \times (n - 1)!$

Each call reduces n by 1, moving steadily toward the base case $n = 0$.

Why this works

Think of it as trusting the function: `factorial(n)` trusts that `factorial(n-1)` already gives the correct answer for the smaller problem — it just multiplies that result by n . This is called the "recursive leap of faith."

```
int factorial(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return n * factorial(n - 1);  
}
```

$factorial(4) \rightarrow 4 \times factorial(3) \rightarrow 4 \times 3 \times factorial(2) \rightarrow \dots \rightarrow 4 \times 3 \times 2 \times 1 \times 1$
 $= 24$

The Call Stack

Tracing factorial(4) — calls pile up, then unwind

1. Calls stack up until the base case

factorial(4) → waiting for factorial(3)

factorial(3) → waiting for factorial(2)

factorial(2) → waiting for factorial(1)

factorial(1) → waiting for factorial(0)

factorial(0) → returns 1 (base case)

2. Results return back up, multiplying as they go

factorial(0) returns 1

factorial(1) returns $1 \times 1 = 1$

factorial(2) returns $2 \times 1 = 2$

factorial(3) returns $3 \times 2 = 6$

factorial(4) returns $4 \times 6 = 24$

Key idea: every recursive call is placed on the call stack and waits, paused, until the call it made returns a value. Once factorial(0) hits the base case, the paused calls resume one by one, from the inside out.

Sum of First N Natural Numbers

Base case: $\text{sum}(0) = 0$

Recursive case: $\text{sum}(n) = n + \text{sum}(n - 1)$

```
int sum(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return n + sum(n - 1);  
}
```

Trace: $\text{sum}(4)$

$\text{sum}(4) = 4 + \text{sum}(3)$

$\text{sum}(3) = 3 + \text{sum}(2)$

$\text{sum}(2) = 2 + \text{sum}(1)$

$\text{sum}(1) = 1 + \text{sum}(0)$

$\text{sum}(0) = 0 \quad \rightarrow \text{base case}$

back-substitute: $1+0=1, 2+1=3, 3+3=6, 4+6=10$

Notice the pattern: this is exactly the same shape as factorial — only the operation changes (+ instead of $\times$).

Fibonacci — Recursion With Two Branches

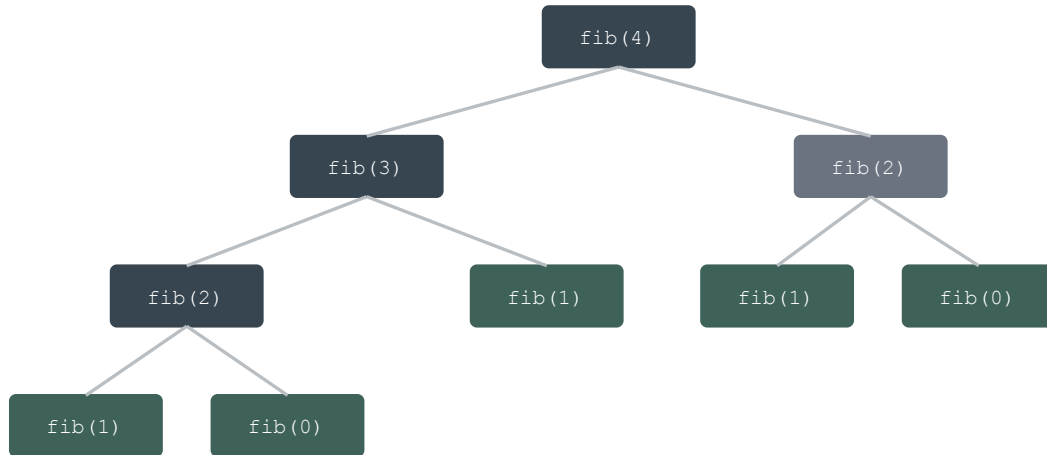
Base cases: $\text{fib}(0)=0$, $\text{fib}(1)=1$

Recursive case:

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n-1) + fib(n-2);  
}
```

Recursion tree for $\text{fib}(4)$ — one call branches into two



Important observation

$\text{fib}(2)$ and $\text{fib}(1)$ are computed more than once — the same work is repeated. This is why plain recursive Fibonacci is slow (exponential time), unlike factorial or sum, which only branch once per call. Binary search, coming next, branches only ONCE per call too — which is why it stays fast.

Recursion vs. Iteration

Aspect	Recursion	Iteration
Repetition mechanism	Function calls itself	Loop (for / while)
State kept in	The call stack (one frame per call)	Loop variables
Code style	Often shorter, mirrors the mathematical / problem definition	Often longer, explicit step-by-step
Memory cost	Extra memory per call — risk of stack overflow if too deep	Constant extra memory
Best suited for	Problems naturally defined in smaller sub-problems (trees, divide-and-conquer, binary search)	Simple repeated counting / accumulation

Common Recursion Mistakes

1 Missing base case

The function never stops calling itself → infinite recursion → "Stack Overflow" crash.

```
int bad(int n) {  
    return n * bad(n - 1);    // no base case!  
}
```

2 Base case never reached

The recursive call doesn't move toward the base case (e.g. $n+1$ instead of $n-1$, or wrong condition).

```
int bad(int n) {  
    if (n == 0) return 1;  
    return n * bad(n + 1);    // grows away  
    from 0  
}
```

3 Wrong / incomplete base case

Base case is missing an edge value the input can actually reach, e.g. only handling $n=0$ but function is called with negative n .

```
int bad(int n) {  
    if (n == 0) return 1;    // n < 0 never  
    handled  
    return n * bad(n - 1);  
}
```

How to Design Any Recursive Function

1

Identify the base case

What is the smallest / simplest input for which you already know the answer without any further computation?

2

Define the recursive case

How can you express the answer for input n using the answer for a smaller input ($n-1$, $n/2$, etc.)?

3

Ensure progress

Check that every recursive call moves strictly closer to the base case — otherwise the recursion never ends.

Preview — Binary Search uses exactly this pattern

Base case: the search range is empty, or the middle element is the target. Recursive case: compare the target with the middle element, then search recursively in only the left half or the right half — the problem size is cut in half every call.

Quick Practice — Try These

- 1 Write a recursive function `power(base, exp)` that computes base^{exp} .
- 2 Write a recursive function to count the number of digits in a positive integer.
- 3 Write a recursive function to check whether a string is a palindrome.
- 4 Write a recursive function `gcd(a, b)` that computes the Greatest Common Divisor of two positive integers using the Euclidean algorithm (Hint: `gcd(a, b) = gcd(b, a % b)`, and `gcd(a, 0) = a`).
- 5 Trace by hand: what does `power(2, 5)` return, and how many calls are made?

Once comfortable with these, we move to Binary Search — a recursive algorithm that cuts the search space in half on every call.

The Euclidean Algorithm — Finding GCD

What is GCD? The Greatest Common Divisor of two numbers a and b is the largest number that divides both of them exactly. Example: $\text{gcd}(48, 18) = 6$.

The key mathematical fact

$$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$$

Any number that divides both a and b must also divide $(a \bmod b)$, because $a \bmod b$ is just a minus some whole-number multiple of b . So the GCD of the pair never changes — only the numbers get smaller.

Base case

$$\text{gcd}(a, 0) = a$$

When b becomes 0, there is nothing left to divide by — a itself is the answer.

Recursive case

$$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$$

Every call replaces (a, b) with $(b, a \bmod b)$ — and $a \bmod b$ is always smaller than b .

Euclidean Algorithm — Code and Trace

```
int gcd(int a, int b) {  
    if (b == 0) {  
        return a;  
    }  
    // base case  
    return gcd(b, a % b);  
    // recursive case  
}
```

Trace: gcd(48, 18)

gcd(48, 18) → 18 does not divide evenly; $48 \bmod 18 = 12$ → call gcd(18, 12)

gcd(18, 12) → $18 \bmod 12 = 6$ → call gcd(12, 6)

gcd(12, 6) → $12 \bmod 6 = 0$ → call gcd(6, 0)

gcd(6, 0) → **b == 0** → **base case, return 6**

gcd(48, 18) = 6. Only 4 calls were needed — notice how quickly the numbers shrink compared to counting down one at a time.

Why the Euclidean Algorithm Is Fast

Naive approach — subtract repeatedly

$\text{gcd}(a, b)$: keep subtracting the smaller from the larger until they're equal.

For $\text{gcd}(1000, 2)$ this takes ~ 500 steps — subtracting 2 over and over.

Time complexity: $O(\max(a, b))$ — slow

Euclidean approach — use mod ($\div$ remainder)

$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$: one mod operation jumps straight to the remainder, skipping every subtraction in between.

Time complexity: $O(\log(\min(a, b)))$ — fast

Connecting back to recursion

Just like factorial and sum, gcd makes exactly ONE recursive call per step — it doesn't branch like Fibonacci. That single-branch shape, combined with the numbers shrinking fast (roughly by a factor each step, not just by 1), is why it finishes in very few calls even for huge inputs. Binary search shares this same fast, single-branch shape — that's exactly why it's efficient too.